Materia: Programación III

UNIDAD 1 A 4 · DEL OBJETO A LA PERSISTENCIA

PARCIAL

Parcial: la librería con despliegue en el VPS

Objetivos

El parcial integra en un único proyecto lo desarrollado en los capítulos 1 a 4: los fundamentos de FastAPI (capítulo 1), los contratos y el CRUD (capítulo 2), la ejecución asincrónica sin bloqueos (capítulo 3) y la persistencia relacional con SQLModel y PostgreSQL, incluidas las relaciones entre entidades (capítulo 4). Y le suma el despliegue: el proyecto se empaqueta en una imagen de Docker, con un `Dockerfile`, y se pone en línea en el VPS. El grupo puede partir de su propio TP entregado.

Al finalizar, el proyecto entregado debe demostrar que el grupo puede:

1. Construir una API asincrónica con FastAPI, con parámetros tipados, dependencias declaradas con `Depends()` y recursos creados en el ciclo de vida de la aplicación.
2. Separar los modelos de tabla de los contratos de entrada y de salida, y responder con los códigos HTTP correctos.
3. Traducir un modelo con asociación, agregación, composición, uno a uno y muchos a muchos a un esquema relacional, justificando de qué lado va cada clave foránea.
4. Declarar la navegación en los dos extremos de cada relación y leerla siempre con carga anticipada, sin caer en el problema N+1.
5. Delegar en la base las garantías que la base puede dar: unicidad, claves foráneas y comprobaciones.
6. Empaquetar el proyecto en una imagen de Docker y ponerlo en línea en el VPS con una configuración que no filtra secretos ni expone lo que no debe.

> **CÓMO SE ENTREGA**
>
> La entrega es **una por grupo**, en formato `.zip`, y la sube un solo integrante al campus, en la tarea de entrega del parcial. El `.zip` va **sin las carpetas** `.venv` **ni** `__pycache__`, y tampoco lleva el archivo `.env`. El detalle de lo que contiene está en el formato de entrega, al final.

QUÉ QUEDA AFUERA

No se evalúan autenticación ni autorización (capítulo 5), repositorios ni Unit of Work (capítulo 6), ni colas de tareas con Redis (capítulos 7 y 8), ni Docker Compose: el despliegue es de una sola imagen, y la base es la que ya tiene el servidor. Tampoco se evalúan las migraciones: el esquema se crea con `create_all` al arrancar la aplicación. La lógica vive en el `service` de cada módulo, que recibe la sesión como parámetro.

Contexto

Una librería de barrio necesita una API para su catálogo y sus ventas. Vende libros de distintas editoriales, clasificados por género y escritos por uno o varios autores, que pueden figurar como autor, coautor, traductor o ilustrador. Registra a sus clientes, que pueden cargar un perfil con teléfono y dirección, y registra cada venta con sus renglones: qué libro, cuántos ejemplares y a qué precio.

El dominio está elegido para que aparezcan todas las relaciones vistas en el capítulo 4, cada una con una razón concreta:

Relación	Entre	Por qué es de ese tipo
Agregación	Libro y editorial	El libro incorpora una editorial que ya existe: viaja su id, y la misma editorial la comparten muchos libros
Agregación opcional	Libro y género	El libro puede incorporar un género que ya existe; es optativo: un libro puede no tenerlo
Muchos a muchos con datos	Libro y autor	Un libro tiene varios autores y un autor varios libros; el rol y el orden son de la relación
Uno a uno	Cliente y perfil	Un cliente tiene como máximo un perfil
Asociación	Venta y cliente	La venta conoce al cliente que compró, pero ninguno es parte del otro
Composición	Venta y renglón	Un renglón no tiene sentido sin su venta: se crea con ella
Asociación	Renglón y libro	El renglón nombra un libro que ya existe

Requisitos técnicos

Componente	Versión mínima	Uso
Python	3.12	Lenguaje

Componente	Versión mínima	Uso
FastAPI	0.111	Framework web; todos los handlers con `async def`
SQLModel	0.0.24	Modelos de tabla y contratos
SQLAlchemy	2.0, asincrónico	Motor, sesión y carga anticipada
asyncpg	0.30	Controlador asincrónico de PostgreSQL
PostgreSQL	16	Base de datos; en el despliegue, la que ya tiene el servidor
python-dotenv	1.0	Lectura del archivo `.env`
uvicorn	El que instala `fastapi[standard]`	Servidor ASGI
Docker	20.10	Construir la imagen y correr el contenedor, en la máquina del grupo y en el VPS

La dirección de la base se lee de la variable de entorno `DATABASE_URL`, definida en un archivo `.env` que no se versiona. El repositorio incluye un `env.example` con la forma del valor y sin ningún secreto (sección 4.13).

Modelo de dominio

Los nombres de tablas y columnas son obligatorios: los casos de prueba y la corrección los usan. Se pueden agregar columnas, pero no quitar ni renombrar las indicadas.

La Figura 1 muestra el modelo como diagrama de clases, con el tipo de cada relación y sus multiplicidades: el rombo lleno es composición, el rombo vacío es agregación y la flecha es asociación. Las tablas que siguen muestran el mismo modelo traducido al esquema, con sus claves y restricciones.

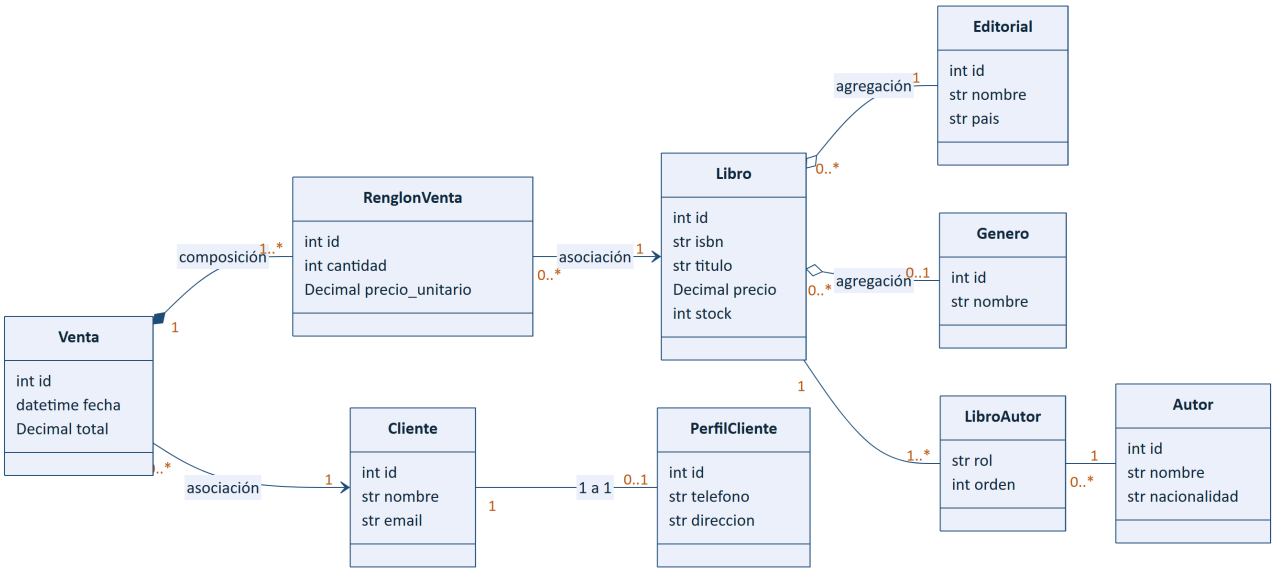


Figura 1 · Diagrama de clases. `LibroAutor` es la clase intermedia del muchos a muchos entre libro y autor, con el rol y el orden de cada autoría.

Tabla	Columnas	Restricciones
`editorial`	`id`, `nombre`, `pais`	`nombre` UNIQUE
`genero`	`id`, `nombre`	`nombre` UNIQUE
`autor`	`id`, `nombre`, `nacionalidad`	Sin restricciones extra
`libro`	`id`, `isbn`, `titulo`, `precio` (NUMERIC 10,2), `stock`, `editorial_id`, `genero_id` (nulo permitido)	`isbn` UNIQUE; CHECK `precio > 0`; CHECK `stock ⩾ 0`
`libro_autor`	`libro_id`, `autor_id`, `rol`, `orden`	Clave primaria (`libro_id`, `autor_id`); CHECK `orden ⩾ 1`; CHECK de `rol` dentro de autor, coautor, traductor, ilustrador
`cliente`	`id`, `nombre`, `email`	`email` UNIQUE
`perfil_cliente`	`id`, `cliente_id`, `telefono`, `direccion`	`cliente_id` UNIQUE
`venta`	`id`, `cliente_id`, `fecha` (con zona horaria), `total` (NUMERIC 12,2)	CHECK `total ⩾ 0`
`renglon_venta`	`id`, `venta_id`, `libro_id`, `cantidad`, `precio_unitario` (NUMERIC 10,2)	CHECK `cantidad > 0`; CHECK `precio_unitario > 0`

Claves foráneas y navegación

Cada clave foránea lleva un índice (PostgreSQL no lo crea solo, sección 4.10) y navegación en los dos extremos con `Relationship(back_populates="...")`.

> **LA TABLA INTERMEDIA ES UNA ENTIDAD**
>
> `libro_autor` se modela como objeto de asociación, con su propia clase, porque tiene datos que no son ni del libro ni del autor: el rol y el orden en la tapa. Una relación muchos a muchos sin clase intermedia (`link_model`) escondería esos datos. Para ir de un libro a sus autores se navega `libro.autorias[i].autor`.

Reglas de negocio

Regla	Enunciado	Garante esperado
RN-01	El ISBN de un libro es único.	UNIQUE en la base; 409
RN-02	El email de un cliente es único.	UNIQUE en la base; 409
RN-03	El precio de un libro es mayor que cero y el stock nunca es negativo.	CHECK en la base y validación del DTO
RN-04	Una venta se crea con sus renglones: los datos de cada renglón viajan dentro del cuerpo del alta de la venta, y no existe un alta de renglones por separado.	DTO anidado; los renglones se agregan a `venta.renglones`
RN-05	Una venta tiene al menos un renglón, y un mismo libro no aparece en dos renglones.	Validación del DTO; 422
RN-06	El precio de cada renglón se copia del libro al registrar la venta y no cambia aunque después cambie el precio del libro.	El servidor lo lee de la base; nunca viaja en el DTO
RN-07	El total de la venta lo calcula el servidor como suma de cantidad por precio unitario.	No viaja en el DTO
RN-08	Las ventas no se modifican: registran un hecho (sección 4.9).	No hay PATCH de ventas
RN-09	Un cliente tiene como máximo un perfil.	UNIQUE en `perfil_cliente.cliente_id`; 409
RN-10	Todo listado se pagina con `limit` (de 1 a 100, por defecto 20) y `offset` (desde 0).	Restricciones en `Query`; 422
RN-11	El listado de libros se puede filtrar por parte del título, sin distinguir mayúsculas.	Parámetro ligado, nunca SQL armado con texto

Regla	Enunciado	Garante esperado
RN-12	Ningún error devuelve al cliente texto interno de la base (nombres de tablas, restricciones o trazas).	Traducción del `IntegrityError` a un mensaje propio

Estructura obligatoria del proyecto

La organización es por módulo funcional, con la misma estructura en todos los módulos:

Ruta	Contenido
`app/main.py`	Aplicación, lifespan y registro de routers
`app/core/config.py`	Lectura de `DATABASE_URL` desde el entorno
`app/core/database.py`	Motor, fábrica de sesiones y dependencia `get_session`
`app/modules/<módulo>/models.py`	Modelos de tabla, restricciones y relaciones
`app/modules/<módulo>/schemas.py`	DTO de entrada (`Create` , `Update`) y de salida (`Read`)
`app/modules/<módulo>/service.py`	Lógica, traducción de errores y lecturas con precarga
`app/modules/<módulo>/router.py`	Rutas HTTP: reciben, delegan y declaran la respuesta
`test/`	Archivos `.http` con los casos de prueba
`Dockerfile` y `.dockerignore`	La imagen del proyecto (ver las consignas de contenedor y despliegue)
`DEPLOY.md`	Cómo se construyó y se corrió la imagen en el VPS

Módulos mínimos: `health` , `editoriales` , `generos` , `autores` , `libros` , `clientes` y `ventas` . `LibroAutor` vive en `libros` , `PerfilCliente` en `clientes` y `RenglonVenta` en `ventas` .

Consignas

Ejercicio 1 · Proyecto, conexión y ciclo de vida

8 puntos

Crear el proyecto con la estructura obligatoria. El motor asincrónico y la fábrica de sesiones se crean en el lifespan de la aplicación, se guardan en `app.state` y el motor se cierra al apagar (sección 1.9). La sesión llega a cada handler con `Depends(get_session)`, una por petición, creada con `expire_on_commit=False` (sección 4.5). La dirección de la base se lee de `.env`. Al arrancar, el lifespan crea las tablas que falten con `SQLModel.metadata.create_all`, con todos los modelos ya importados. `create_all` sólo crea las tablas que no existen y no altera las que ya existen: un cambio en un modelo no llega a una base ya creada, y por eso en un proyecto real el esquema se versiona con migraciones (sección 4.11), que acá no se evalúan. Se expone `GET /health/live`, que no toca la base, y `GET /health/ready`, que ejecuta `SELECT 1`.

Ejercicio 2 · Modelos, restricciones y relaciones

22 puntos

Escribir los modelos de tabla del modelo de dominio con todas sus restricciones (UNIQUE, CHECK, claves foráneas e índices). Declarar la navegación en los dos extremos de cada clave foránea. `Cliente.perfil` devuelve un objeto y no una lista. El dinero se representa con `Decimal`, nunca con `float`.

Ejercicio 3 · Contratos y CRUD

12 puntos

Implementar los endpoints de referencia con DTO separados de los modelos de tabla. Los DTO de alta no llevan `id`. Los PATCH aplican sólo los campos enviados (`exclude_unset`). Los códigos de respuesta son 201 al crear, 404 si el recurso no existe, 409 si la base rechaza la escritura y 422 si el cuerpo o los parámetros no cumplen el contrato. Los listados se paginan (RN-10) y la búsqueda por título usa parámetros ligados (RN-11).

Ejercicio 4 · Altas con relaciones: ids y datos

17 puntos

El alta de un libro recibe los ids de cosas que ya existen: `editorial_id`, `genero_id` y la lista `autores`, donde cada elemento trae `autor_id` más los datos de la relación (`rol` y `orden`). El alta de una venta recibe `cliente_id` (asociación: viaja un id) y la lista `renglones` con `libro_id` y `cantidad` (composición: viajan los datos de la parte, que nace con la venta). La venta se crea con sus renglones y cumple RN-04 a RN-07: el precio de cada renglón se copia del libro y el total lo calcula el servidor. En el alta del libro ningún `if` consulta antes si un id existe: se escribe, y si la base rechaza la clave foránea, se responde 409.

Ejercicio 5 · Lecturas sin N+1

8 puntos

Toda respuesta que incluya datos de una relación los obtiene con carga anticipada declarada en la misma consulta: `joinedload` para relaciones a uno y `selectinload` para colecciones (sección 4.6). El detalle de un libro (con editorial, género y autores) y el detalle de una venta (con cliente y renglones) se resuelven con un número fijo de consultas, que no crece con la cantidad de autores o de renglones. Ninguna lectura puede depender de la carga perezosa.

Ejercicio 6 · Asincronía sin bloqueos

8 puntos

Todos los handlers son `async def` y ninguno ejecuta una llamada bloqueante (`time.sleep`, `requests`, acceso síncrono a la base). Al registrar una venta se agenda con `BackgroundTasks` el aviso del comprobante al cliente, simulado con `await asyncio.sleep(0.5)` y un mensaje en el log, que corre después de enviar la respuesta. Mientras corre el aviso, el servidor sigue atendiendo otras peticiones.

Ejercicio 7 · Contenedor: la imagen

15 puntos

El proyecto del parcial se empaqueta en una **imagen de Docker**, construida con un `Dockerfile` y puesta a correr con `docker run`. No se usa Docker Compose: la base de datos no forma parte del paquete, porque el servidor ya tiene PostgreSQL. La lógica de la aplicación no cambia: lo que se agrega son el `Dockerfile` y el `.dockerignore`. Si el proyecto no arranca en el contenedor por un defecto propio (por ejemplo, una ruta de salud que no responde como pide este documento), se corrige y se anota en `DEPLOY.md`. Esta consigna es lo que se entrega en el repositorio; la siguiente, lo que tiene que estar en línea.

La imagen. El `Dockerfile` parte de `python:3.12-alpine`. La razón del límite de 100 MB es concreta: el VPS descarga y guarda esa imagen, cada megabyte de la base se paga en disco y en tiempo de descarga en cada despliegue, y cada paquete del sistema que la imagen trae es superficie de ataque que la aplicación nunca usa. Medida con Docker 29.2 el 2 de noviembre de 2026, `python:3.12-alpine` ocupa **74,7 MB** en disco; `python:3.12-slim`, la alternativa habitual, ocupa **180 MB** y por eso no cumple. El límite es de la **imagen base**: la del proyecto, con sus dependencias instaladas, es bastante más grande, y eso es lo esperado. Alpine usa musl en lugar de glibc, y eso suele traer problemas con paquetes que se compilan al instalarse; se verificó que las dependencias de este stack (asyncpg, pydantic-core y el resto del `requirements.txt`) se instalan en esa imagen sin compilar nada, así que no hace falta instalar ningún compilador.

El `Dockerfile` cumple lo siguiente, y cada punto tiene su razón:

- Copia y instala `requirements.txt` **antes** de copiar el código, con `--no-cache-dir`: mientras las dependencias no cambien, esa capa sale de la caché y un cambio en el código no reinstala todo.
- La aplicación **no corre como root**: se crea un usuario sin privilegios y se lo activa con `USER`. Si alguien vulnera la aplicación, no es dueño del contenedor.
- **La imagen no contiene secretos.** El `.dockerignore` excluye `.env`, `.venv`, `.git` y los `__pycache__`, y ni la dirección de la base ni su contraseña aparecen en el `Dockerfile`: entran como variables de entorno al correr el contenedor. Lo que se hornea en una capa queda ahí para siempre, aunque una capa posterior lo borre.
- Al arrancar, el contenedor levanta uvicorn con `app.main:app` escuchando en `0.0.0.0` (dentro de un contenedor, `127.0.0.1` sólo se alcanza desde el propio contenedor, y el puerto publicado nunca respondería). La aplicación crea sus tablas al arrancar, y si la base no responde el arranque falla y el contenedor termina con error: es lo que se quiere, porque una aplicación que no llega a su base no debería quedar atendiendo.
- Declara un `HEALTHCHECK` sobre `GET /health/ready`. Sin Compose, es la propia imagen la que dice cuándo está sana, y `docker ps` lo muestra.

Cómo se corre. La imagen se pone a andar con un comando de esta forma:

```
docker run -d --name libreria -p <puerto>:8000 --restart unless-stopped \
  --env-file <archivo-de-variables> libreria
```

- `-p <puerto>:8000` publica **sólo la aplicación**, en el puerto que asigna la cátedra.
- `--restart unless-stopped` hace que Docker vuelva a levantar el contenedor si termina: por ejemplo, si el servidor se reinicia, o si la base todavía no respondía cuando arrancó.
- `--env-file` entrega la configuración desde un archivo que vive **en el servidor**, fuera del repositorio y de la imagen. Tiene `DATABASE_URL` —con el driver `postgresql+asyncpg`, y el servidor, el usuario, la contraseña y el nombre de la base que provee la cátedra— y la variable que activa el diagnóstico de consultas (`SQL_ECHO` en el ejemplo del `env.example`) en `false`, que deja de escribir cada consulta en el registro.

> **LA BASE NO ESTÁ EN `LOCALHOST`**
>
> Dentro de un contenedor, `localhost` es el propio contenedor y no el servidor: una `DATABASE_URL` con `localhost` falla con «Connection refused» aunque PostgreSQL esté funcionando. Si la base corre directamente en el servidor, el contenedor la alcanza por el nombre `host.docker.internal`, que en Linux hay que habilitar con `--add-host=host.docker.internal:host-gateway` en el `docker run`. Si la cátedra entrega otra dirección, se usa esa.

> **EL HEALTHCHECK Y `LOCALHOST`**
>
> Dentro de una imagen Alpine no hay `curl`; `wget`, que sí viene, resuelve `localhost` primero a `::1` (IPv6), y uvicorn con `--host 0.0.0.0` escucha sólo en IPv4. Un `HEALTHCHECK` con `http://localhost:8000/...` falla con «Connection refused» aunque el servidor responda perfectamente desde afuera, y el contenedor queda `unhealthy` sin que nada parezca roto. Se comprueba con `docker exec libreria wget -qO- http://127.0.0.1:8000/health/live`, y la dirección que sirve es `127.0.0.1`.

Ejercicio 8 · Despliegue en el VPS

10 puntos

El proyecto se despliega en el VPS con los datos de acceso y la base que provee la cátedra. El grupo construye la imagen en el servidor —o la construye en otro lado y la lleva — y la corre con `docker run`; el grupo elige y lo documenta. Queda accesible desde afuera en la URL que se declara en `DEPLOY.md`. Una vez arriba, la base se carga ejecutando dentro del contenedor el comando de datos iniciales del proyecto (`docker exec libreria …`). El despliegue **tiene que seguir en línea hasta que se publiquen las notas**, porque la corrección lo prueba.

`DEPLOY.md` incluye: la URL pública, el día y la hora del despliegue, los comandos `docker build` y `docker run` completos —con el nombre del archivo de variables y sin su contenido—, cualquier cambio que se le haya hecho al proyecto para que corra en el contenedor, y cómo se actualiza la aplicación: se construye la imagen de nuevo, se elimina el contenedor y se lo vuelve a correr. Los datos no se pierden, porque viven en la base del servidor y no en el contenedor.

Endpoints de referencia

Método	Ruta	Cuerpo	Respuesta	Códigos
GET	`/health/live`	Sin cuerpo	`{"estado": "vivo"}`	200
GET	`/health/ready`	Sin cuerpo	Estado de la base	200
POST	`/editoriales/`	`EditorialCreate`	`EditorialRead`	201, 409, 422
GET	`/editoriales/`	`limit`, `offset`	Lista de `EditorialRead`	200, 422
GET	`/editoriales/{id}`	Sin cuerpo	`EditorialRead`	200, 404, 422
PATCH	`/editoriales/{id}`	`EditorialUpdate`	`EditorialRead`	200, 404, 409
GET	`/editoriales/{id}/libros`	Sin cuerpo	Libros de la editorial	200, 404
POST	`/generos/`	`GeneroCreate`	`GeneroRead`	201, 409
GET	`/generos/`	Sin cuerpo	Lista de `GeneroRead`	200
GET	`/generos/{id}/libros`	Sin cuerpo	Libros del género	200, 404

Método	Ruta	Cuerpo	Respuesta	Códigos
POST	`/autores/`	`AutorCreate`	`AutorRead`	201, 422
GET, PATCH	`/autores/{id}`	`AutorUpdate` en PATCH	`AutorRead`	200, 404, 422
GET	`/autores/{id}/libros`	Sin cuerpo	Libros con rol y orden	200, 404
POST	`/libros/`	`LibroCreate` con `autores`	Detalle del libro	201, 409, 422
GET	`/libros/`	`limit`, `offset`, `titulo`, `genero_id`	Libros con editorial y género	200, 422
GET	`/libros/{id}`	Sin cuerpo	Libro con editorial, género y autores	200, 404
PATCH	`/libros/{id}`	`LibroUpdate`	Detalle del libro	200, 404, 409, 422
POST	`/clientes/`	`ClienteCreate`	`ClienteRead`	201, 409, 422
GET	`/clientes/{id}`	Sin cuerpo	Cliente con su perfil	200, 404
PATCH	`/clientes/{id}`	`ClienteUpdate`	Cliente con su perfil	200, 404, 409, 422
POST	`/clientes/{id}/perfil`	`PerfilCreate`	`PerfilRead`	201, 404, 409
GET	`/clientes/{id}/ventas`	Sin cuerpo	Ventas del cliente	200, 404
POST	`/ventas/`	`VentaCreate` con `renglones`	Venta con cliente y renglones	201, 409, 422
GET	`/ventas/`	`limit`, `offset`	Lista de ventas	200, 422
GET	`/ventas/{id}`	Sin cuerpo	Venta con cliente y renglones	200, 404

Ejemplo del cuerpo de `POST /ventas/`. Viaja el id del cliente y los datos de cada renglón; no viajan el precio, el total ni la fecha:

```
{
  "cliente_id": 2,
  "renglones": [
    { "libro_id": 1, "cantidad": 2 },
    { "libro_id": 4, "cantidad": 1 }
  ]
}
```

Respuesta esperada, con el precio copiado de cada libro y el total calculado por el servidor:

```
{
  "id": 2,
  "fecha": "2026-10-08T21:14:11.751581Z",
  "total": "77000.00",
  "cliente": { "id": 2, "nombre": "Tomás Ríos", "email": "tomas@example.com" },
  "renglones": [
    { "libro_id": 1, "titulo": "Rayuela", "cantidad": 2, "precio_unitario":
"25000.00", "subtotal": "50000.00" },
    { "libro_id": 4, "titulo": "Respiración artificial", "cantidad": 1,
"precio_unitario": "27000.00", "subtotal": "27000.00" }
  ]
}
```

Ejemplo del cuerpo de `POST /libros/`. Viajan ids de editorial, género y autores, más los datos propios de cada autoría:

```
{
  "isbn": "978-950-00-0006-6",
  "titulo": "Los que aman, odian",
  "precio": "21000.00",
  "stock": 4,
  "editorial_id": 1,
  "genero_id": 1,
  "autores": [
    { "autor_id": 3, "rol": "coautor", "orden": 1 },
    { "autor_id": 4, "rol": "coautor", "orden": 2 }
  ]
}
```

Requisitos no funcionales

Requisito	Detalle
Asincronía	Todos los handlers con `async def`; ninguna llamada bloqueante dentro de ellos
Credenciales	Ninguna contraseña en el código, en el repositorio ni en la imagen

Requisito	Detalle
Carga perezosa	Prohibida: toda relación que se lee se precarga en la consulta
SQL	Nunca armado por concatenación de texto con datos del usuario
Errores	Mensajes propios; nunca el texto de la base ni una traza
Unicidad	Garantizada por la base; no se verifica con una consulta previa
Diagnóstico	Las consultas emitidas se pueden ver en la consola (por ejemplo, `echo=True` configurable)
Contenedor	Imagen base `python:3.12-alpine` ; la aplicación no corre como root; ningún secreto dentro de la imagen

Casos de prueba mínimos

El proyecto incluye un script o comando que deja la base en un estado inicial conocido, y archivos `.http` en `test/` que demuestran, como mínimo, los siguientes casos. Cada caso indica la respuesta esperada en un comentario. Los casos D se demuestran sobre el proyecto desplegado en el VPS, con los comandos y las salidas guardados en la carpeta `evidencia/` .

Caso	Qué se demuestra	Resultado esperado
C-01	Un parámetro de ruta que no es entero	422, sin ejecutar el handler
C-02	ISBN o email repetido	409 con mensaje propio
C-03	Segundo perfil para el mismo cliente	409
C-04	Libro con una editorial o un autor inexistente	409 con mensaje propio
C-05	Venta con un libro o un cliente inexistente	409
C-06	Venta válida con dos renglones	201, con los renglones creados, los precios copiados y el total calculado
C-07	Detalle de un libro y de una venta	Cantidad fija de consultas en la consola, la misma con uno o con varios autores o renglones
C-08	Búsqueda por título con el texto `' OR '1'='1`	200 con lista vacía
D-01	`docker images` con la imagen base y con la del proyecto	`python:3.12-alpine` por debajo de los 100 MB, y el `Dockerfile` parte de ella
D-02	Servidor limpio: `docker build` y `docker run` ; después `docker ps`	El contenedor en estado `healthy` ; en `docker logs` , el arranque completo, con las tablas ya creadas

Caso	Qué se demuestra	Resultado esperado
D-03	Desde afuera del VPS, en la URL pública: `GET /health/ready`, `GET /libros/` con los datos iniciales cargados y `POST /ventas/` con el cuerpo de ejemplo del caso C-06	200; 200 con la lista de libros; 201 con los precios copiados de cada libro y el total calculado
D-04	`docker rm -f libreria` y el mismo `docker run` otra vez	Vuelve a estar `healthy`, los datos siguen —la base no vive en el contenedor— porque `create_all` sólo crea las tablas que faltan
D-05	Correr la imagen sin `DATABASE_URL`	El contenedor no queda sirviendo: termina con un error, porque el proyecto no trae ninguna dirección de base por defecto
D-06	Usuario del proceso	`docker exec libreria id` no es root
D-07	La venta registrada en el caso D-03	El aviso del comprobante aparece en `docker logs libreria` después de la línea de la respuesta

Criterios de evaluación

Criterio	Peso	Qué se mira
Proyecto, conexión y ciclo de vida	8 %	Estructura, lifespan, creación de las tablas al arrancar, `Depends`, `.env` y `env.example`
Modelos, restricciones y relaciones	22 %	Lado de cada clave foránea, uno a uno con UNIQUE, objeto de asociación, CHECK, índices, navegación en los dos extremos
Contratos y CRUD	12 %	DTO separados, códigos HTTP, PATCH parcial, paginación, parámetros ligados
Altas con relaciones	17 %	Ids en agregación y asociación, datos de los renglones dentro de la venta en la composición
Lecturas sin N+1	8 %	Carga anticipada adecuada a cada relación; número fijo de consultas
Asincronía sin bloqueos	8 %	Handlers asincrónicos sin llamadas bloqueantes; aviso en segundo plano con `BackgroundTasks`
Contenedor: la imagen	15 %	Imagen base `python:3.12-alpine`, capas en orden, usuario sin privilegios, sin secretos en la imagen, `HEALTHCHECK`, y ejecución con `docker run` y las variables por `--env-file`

Criterio	Peso	Qué se mira
Despliegue en el VPS	10 %	El proyecto en línea en la URL declarada, `DEPLOY.md` completo, los casos D con su evidencia

Un proyecto que no levanta, que no crea su esquema al arrancar sobre una base vacía o que contiene credenciales reales en el repositorio, en la imagen o en el zip no se corrige hasta que se subsane. Si el despliegue no está accesible en el momento de la corrección, el criterio de despliegue en el VPS se evalúa sólo sobre lo que consta en `DEPLOY.md` y en la carpeta `evidencia/`, y los casos D que no se puedan comprobar no suman.

Formato de entrega

La entrega es **una por grupo**, en **un único archivo** `.zip` que sube **un solo integrante** al campus, en la tarea de entrega del parcial. El archivo se llama `Parcial_Cap1a4_<NombreDelGrupo>.zip`; la fecha límite se publica en el campus.

El `.zip` contiene el proyecto en una sola carpeta:

- El código fuente completo, `requirements.txt`, `env.example` (con la forma de `DATABASE_URL`, sin valores reales), la carpeta `test/`, el `Dockerfile` y el `.dockerignore`.
- `README.md` con los nombres y apellidos de los integrantes y los pasos para levantar el proyecto en la máquina del grupo desde cero: entorno virtual, dependencias, `.env`, arranque del servidor (que crea las tablas) y carga de datos iniciales.
- `DEPLOY.md` con lo que pide la consigna de despliegue.
- La carpeta `evidencia/` con la captura o el fragmento de consola del caso C-07 (las consultas del detalle de un libro y de una venta) y la salida de cada caso D.

El `.zip` **no contiene** las carpetas `.venv` ni `__pycache__` ni el archivo `.env`: el entorno virtual se reconstruye con `requirements.txt`, y los `__pycache__` los genera Python solo. Si incluye la carpeta `.git`, su historial también se revisa: una contraseña que alguna vez se subió y se borró después sigue estando en el historial, y en ese caso se pide además cambiarla.